Parallel Streams: When They Help and When They Hurt

8 min read Updated 28 Jul 2026 Topic 34 of 65

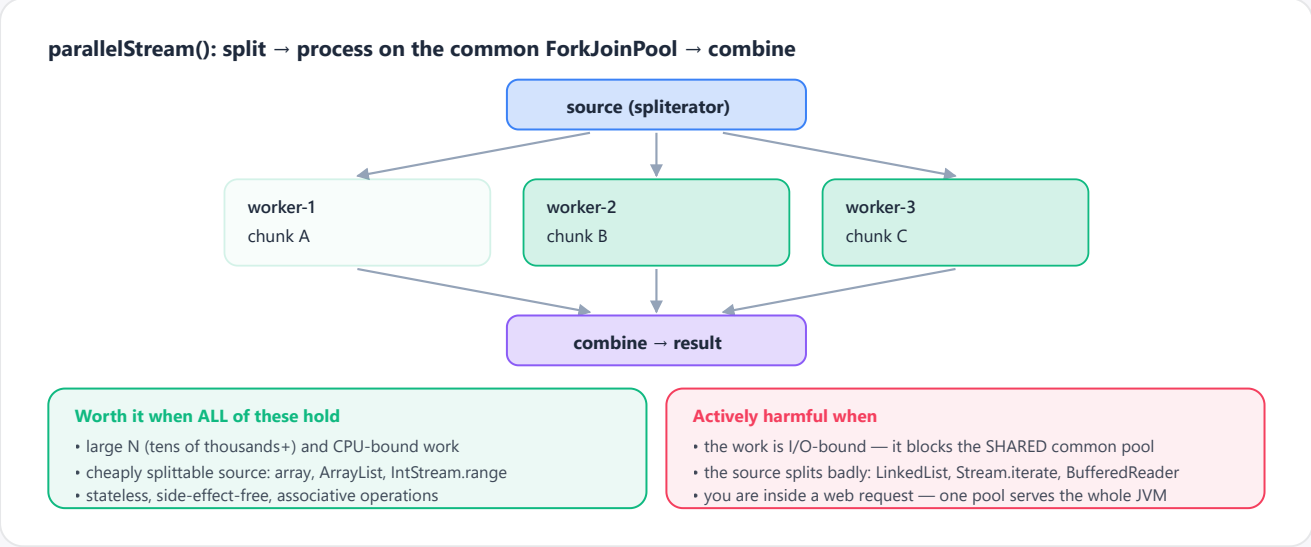
Bookmark Mark done

Why it matters

`.parallelStream()` is the most dangerous easy win in Java. It is one word, it looks like free performance, and in a web application it is usually either slower or actively harmful — because every parallel stream in the JVM shares one thread pool. Interviewers ask precisely because the naive answer ("use it for big collections") misses the two things that actually matter: what the work is doing, and which pool it runs on.

The concept, simply

A parallel stream splits the source into chunks, processes them on multiple threads, and combines the results.



Split, process, combine. Every step has a cost, and the whole thing only pays off when the per-element work is large enough to dwarf that overhead.

The critical detail: unless you do something special, **every parallel stream in the JVM runs on the same** `ForkJoinPool.commonPool()`, whose size defaults to the number of cores minus one. It is a shared resource, and one slow pipeline starves every other.

How it works

Parallelism pays only when several conditions hold together:

- **Enough elements and enough work per element.** The rule of thumb is $N \times Q$ — element count times cost per element — needs to be large. Ten thousand trivial operations is not enough; a thousand expensive ones might be.
- **A cheaply splittable source.** Arrays, `ArrayList` and `IntStream.range` split in O(1) by index. `LinkedList`, `Stream.iterate` and I/O-backed streams split badly or not at all, and the overhead swamps any gain.
- **Stateless, side-effect-free, associative operations.** A reduction that is not associative gives a different answer depending on how the work was split.
- **CPU-bound work.** This is the big one.

Blocking I/O in a parallel stream is the classic production incident. The common pool has a handful of threads; making HTTP calls inside a parallel stream ties them all up waiting on the network. Every other parallel stream in the application — including ones in library code you did not write — stops. And parallelism buys nothing anyway, because the bottleneck is the network, not the CPU.

Code examples

Where it genuinely helps:

JAVA

Copy

```java
// ✓ Large N, CPU-bound work, array-backed source, associative reduction.
long primes = IntStream.rangeClosed(2, 10_000_000)
                       .parallel()
                       .filter(this::isPrime)     // expensive, pure
                       .count();

// ✓ Also fine — a big array and real computation per element.
double[] results = Arrays.stream(bigArray)
```

Where it hurts, in the order people hit it:

JAVA Copy

```java
// ✗ 1. Blocking I/O — starves the SHARED common pool for the whole JVM.
urls.parallelStream().map(this::httpGet).toList();
// Every other parallel stream in the application now waits behind these.
// Use a CompletableFuture with its own executor instead (Chapter 8).

// ✗ 2. Tiny collection — setup cost exceeds the work.
List.of("a", "b", "c").parallelStream().map(String::toUpperCase).toList();

// ✗ 3. Badly splittable source.
Stream.iterate(1, n → n + 1).limit(1_000_000).parallel()...
// iterate is inherently sequential — each element depends on the previous

// ✗ 4. Shared mutable state — a race, and it will pass your tests.
List<String> results = new ArrayList<>();
list.parallelStream().forEach(results::add);      // ArrayList is not thre
// Symptoms: missing elements, nulls, or ArrayIndexOutOfBoundsException.

// ✓ Collectors handle parallelism correctly — that is what they are for.
List<String> results = list.parallelStream().map(this::transform).toList()
```

Ordering, which costs more than people expect:

JAVA Copy

```java
// forEach gives NO order guarantee in parallel — that is the point of it.
list.parallelStream().forEach(System.out::println);       // interleaved

// forEachOrdered restores order, and serialises the terminal stage —
// often cancelling out the parallelism you just paid for.
list.parallelStream().forEachOrdered(System.out::println);

// findFirst must respect encounter order and is therefore more expensive
```

Escaping the common pool, which is the answer to "what if I must?":

JAVA Copy

```java
// A parallel stream submitted INSIDE a custom ForkJoinPool runs on that p
// It is an undocumented-but-stable behaviour, and widely used.
ForkJoinPool pool = new ForkJoinPool(4);
try {
    List<Result> results = pool.submit(() ->
        items.parallelStream().map(this::process).toList()
    ).get();
} finally {
    pool.shutdown();
}
// Honestly: if you need this level of control, CompletableFuture with an
// explicit executor expresses the intent better than a parallel stream.
```

Under the hood

Splitting is done by the source's `Spliterator`, which reports both an estimated size and characteristics like `SIZED` and `SUBSIZED`. A source that knows its size and can split by index — an array or `ArrayList` — produces balanced chunks cheaply. A `LinkedList` must traverse to split, and `Stream.iterate` cannot meaningfully split at all, which is why those sources perform so badly.

The pool uses **work stealing**: each worker has its own deque, and an idle worker steals from a busy one's tail. That handles uneven chunk sizes well, and it is also why a blocking task is so damaging — a blocked thread cannot be stolen from usefully, and the pool has no spare capacity by design.

Associativity matters for correctness, not just speed. A reduction that is not associative produces a different result depending on how the work was split, so the same code can give different answers on different machines or runs. Subtraction is the

The honest summary to give in an interview: default to sequential; reach for parallel only when profiling shows a CPU-bound bottleneck over a large, array-backed source; and measure, because the overhead frequently exceeds the gain. In a request-handling service, the right answer is usually not a parallel stream at all — it is an explicit executor sized for the workload.

Common mistakes

- **Blocking I/O in a parallel stream.** Starves the shared common pool for the whole JVM and gains nothing.
- **Parallelising small collections.** Setup cost exceeds the work.
- **Mutating shared state in `forEach`.** A race that passes tests and fails in production. Use a collector.
- **Non-associative reductions.** Different answers on different runs.
- **Using `findFirst` where `findAny` would do.** Order preservation costs real time in parallel.
- **Assuming more threads means faster.** The common pool is sized to the cores; contention and merging are not free.
- **Parallelising a `LinkedList` or `Stream.iterate`.** They split badly or not at all.
- **Not measuring.** This is the one area where intuition is reliably wrong.

Interview lens

> **Q: "When would you use a parallel stream?"**
> A: Rarely, and only when four things hold: the work is CPU-bound, there are enough elements that N times the per-element cost is large, the source splits cheaply — an array or `ArrayList`, not a `LinkedList` — and the operations are stateless and associative. Then I measure, because the split-and-merge overhead often exceeds the gain.

A: `ForkJoinPool.commonPool()` by default, sized to available cores minus one, and shared by the entire JVM. That is the single most important fact about them: one slow parallel stream starves every other, including ones inside libraries. You can run a pipeline on a different pool by submitting it to your own `ForkJoinPool`, but at that point an explicit executor with `CompletableFuture` is usually the clearer design.

Q: "Why is blocking I/O in a parallel stream bad?"

A: Two reasons. It ties up the shared common pool's small number of threads waiting on the network, so unrelated parallel work across the application stalls. And it buys nothing, because the bottleneck is I/O rather than CPU — the right tool is asynchronous calls on an executor sized for concurrency, not parallelism sized for cores.

Q: "Is `parallelStream().forEach()` safe?"

A: Only if the action is thread-safe and you do not care about order. Adding to an `ArrayList` from it is a race — you get missing elements, nulls, or an index exception, and it will pass a small test. The correct approach is `collect` or `toList()`, because collectors are written to combine per-thread containers safely.

Q: "Why must a reduction be associative?"

A: Because the stream is split arbitrarily and the partial results are combined in an unspecified order. If the operation is not associative — subtraction, say — the answer depends on how the work happened to be divided, so the same code gives different results on different machines. The identity must be a genuine identity too, or it is applied once per chunk.

A: `findAny` in parallel, whenever any match will do. `findFirst` must respect encounter order, which forces coordination between the workers and gives up much of the parallel benefit. Sequentially they behave the same.

Cheat-card

- All parallel streams share `ForkJoinPool.commonPool()` — cores − 1, JVM-wide.
- Needs: **CPU-bound · large N × Q · splittable source · stateless, associative**.
- **Never do blocking I/O** in one — it starves the whole application.
- Bad sources: `LinkedList`, `Stream.iterate`, I/O-backed streams.
- Shared mutable state in `forEach` = race. **Use collectors.**
- `findAny` > `findFirst`; `forEachOrdered` serialises the terminal stage.
- Non-associative reduction ⟶ different answers per run.
- Own pool: `new ForkJoinPool(n).submit(() → …parallelStream()…)`.
- **Default to sequential. Measure before parallelising.**

QUICK CHECK — no marks, just for you.

1. Which thread pool do parallel streams use by default?

- ○ ForkJoinPool.commonPool() — shared by the entire JVM and sized to available cores minus one
- ○ The thread that created the stream, sequentially
- ○ A new pool per stream
- ○ The servlet container's request pool

Check answer

- IntStream.range(0, 1_000_000)
- Stream.iterate — each element depends on the previous, so it cannot be split
- An int[] array
- An ArrayList

Check answer

3. list.parallelStream().forEach(results::add) where results is an ArrayList. What is wrong?

- It will always throw ConcurrentModificationException
- It works but is slower than a sequential forEach
- ArrayList is not thread-safe, so this races — you get missing elements, nulls or an index exception, and it will pass a small test
- Nothing — forEach is synchronised internally

Check answer

‹ Previous
Collectors, groupingBy and Reduction

Next ›
Optional Done Right

LEARN MORE

ARTICLES & DOCS

ForkJoinPool — API docs
docs.oracle.com

VIDEOS

"parallel streams" — video explainers
YouTube

parallel streams interview questions
YouTube

Hand-picked references for this lesson.